P.Vishwa nandan(192473015)

TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN (5 MARKS)

1. Aim

To identify repeated database-access operations in StudentDB, CourseDB, and MarksDB and redesign them using the Data Access Object (DAO) design pattern.

The application uses a generic DAO interface to separate database operations from business logic and improve code reusability, maintainability, and testing.

2. Problem Description

A student project contains three classes:

- StudentDB
- CourseDB
- MarksDB

Each class performs almost identical database operations.

These operations include:

1. Opening a database connection.
2. Creating an SQL query.
3. Executing the query.
4. Reading the ResultSet.
5. Converting database records into Java objects.
6. Closing database resources.

However, the three classes use different database tables and different fields.

This creates code duplication and makes the application difficult to maintain.

The solution is to introduce a generic DAO interface.

3. Task A3.1 – Identify the Common Pattern (2 Marks)

3.1 Repeated Database Operations

All three classes perform the following common operations.

Step 1 – Establish Database Connection

Each class connects to the database using JDBC.

Connection con = DriverManager.getConnection(

URL, USER, PASSWORD

);

Step 2 – Create SQL Query

Each class creates SQL statements for database operations.

Example:

String sql = "SELECT * FROM students WHERE student_id = ?";

Step 3 – Execute the Query

The SQL statement is executed using Statement or PreparedStatement.

PreparedStatement ps = con.prepareStatement(sql);

Step 4 – Read ResultSet

The result of a SELECT query is retrieved using ResultSet.

ResultSet rs = ps.executeQuery();

Step 5 – Convert Database Records into Objects

The retrieved database values are assigned to Java object fields.

For example, a student table record is converted into a Student object.

Step 6 – Close Database Resources

The connection, PreparedStatement, and ResultSet are closed after the operation.

Try-with-resources can be used for automatic resource management.

3.2 Common and Different Parts

Common Operations	**Different Operations**
Opening database connection	Database table name
Creating SQL statements	Column names
Executing SQL queries	Java object type
Reading ResultSet	Primary key field
Converting records into objects	Fields and data types
Closing resources	Table-specific business rules

3.3 Problem With the Existing Design

The same database connection and query-execution code is repeated in multiple classes.

This causes:

1. Code duplication.
2. Difficult maintenance.
3. More places for programming errors.
4. Repeated modifications when database logic changes.
5. Difficulty in testing database operations independently.

3.4 Proposed Solution – DAO Design Pattern

The DAO (Data Access Object) design pattern separates database operations from the business logic of an application.

A generic DAO interface defines common CRUD operations.

CRUD means:

- Create – Insert a record.
- Read – Find or retrieve records.
- Update – Modify a record.
- Delete – Remove a record.

Each database entity implements the same interface with its own table-specific SQL queries.

This avoids repeating the overall database-access structure.

4. Task A3.2 – Design a Generic Data-Access Interface (2 Marks)

4.1 Generic DAO Interface

The generic interface is defined as:

```
public interface Dao<T, K> {

    void insert(T entity) throws SQLException;

    Optional<T> findById(K id) throws SQLException;

    List<T> findAll() throws SQLException;

    boolean update(T entity) throws SQLException;
```

```
    boolean deleteById(K id) throws SQLException;
}
```

Explanation

The interface uses two generic types:

T – Entity Type

Represents the Java object being stored.

Examples:

- Student
- Course
- Marks

K – Primary Key Type

Represents the data type of the entity's primary key.

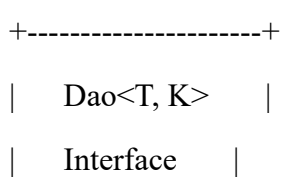
Examples:

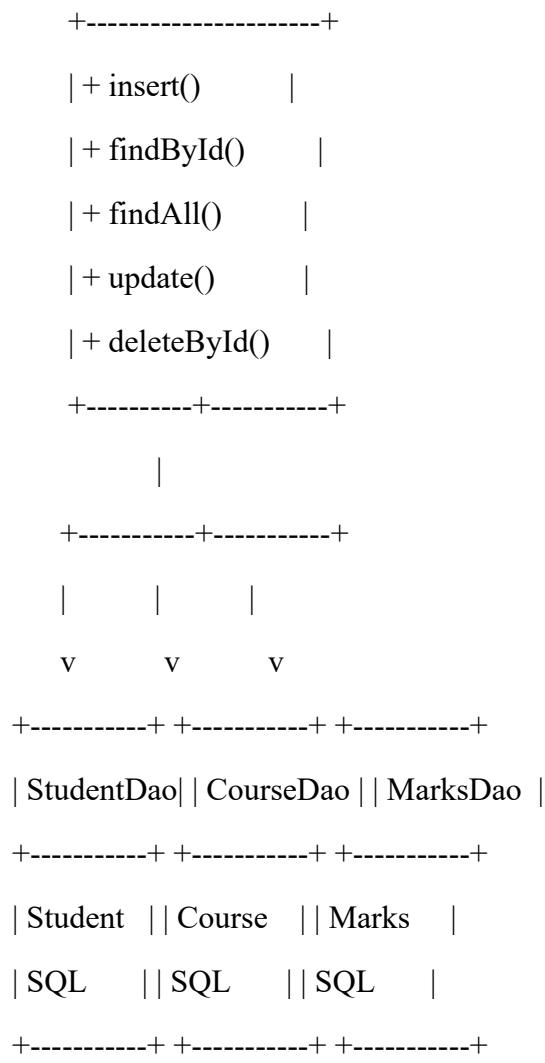
- Integer
- Long
- String

The interface defines five common database operations.

Method	Purpose
insert(T entity)	Inserts a new record
findById(K id)	Finds a record using its primary key
findAll()	Retrieves all records
update(T entity)	Updates an existing record
deleteById(K id)	Deletes a record using its primary key

4.2 Generic DAO Class Diagram





All three DAO classes implement the common interface but contain their own table-specific SQL statements.

4.3 SQL Table for Student

Before implementing StudentDao, create a student table in MySQL.

```
CREATE DATABASE IF NOT EXISTS student_db;


USE student_db;


CREATE TABLE students (


    student_id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
    register_no VARCHAR(20) NOT NULL UNIQUE,

    student_name VARCHAR(100) NOT NULL,

    email VARCHAR(150) NOT NULL
);
```

Explanation:

- student_id is the primary key.
- register_no stores the student's register number.
- student_name stores the student's name.
- email stores the student's email address.
- AUTO_INCREMENT automatically generates a new student ID.

4.4 Java Program 1 – Dao.java

This is the generic data-access interface.

```
import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public interface Dao<T, K> {

    void insert(T entity) throws SQLException;

    Optional<T> findById(K id) throws SQLException;

    List<T> findAll() throws SQLException;

    boolean update(T entity) throws SQLException;

    boolean deleteById(K id) throws SQLException;
```

}

Explanation

The interface defines the common database operations.

It does not contain table-specific SQL queries.

The concrete DAO classes provide the implementation.

4.5 Java Program 2 – DatabaseConnection.java

Explanation

1. The URL identifies the MySQL database.
2. USER stores the database username.
3. PASSWORD stores the database password.
4. getConnection() creates and returns a JDBC connection.
5. The DAO classes use this method whenever they need database access.

Update the username and password according to your MySQL configuration.

4.6 Java Program 3 – Student.java

Explanation

The Student class contains four fields:

- studentId
- registerNo
- studentName
- email

The constructor initialises the student details.

Getter and setter methods allow access to and modification of the fields.

4.7 Java Program 4 – StudentDao.java

Explanation

The StudentDao class implements the generic Dao interface.

It provides five database operations.

1. insert()

Inserts a student record into the students table.

It also retrieves the automatically generated primary key using getGeneratedKeys().

2. findById()

Searches for a student using the student ID.

It returns an Optional containing the student if a record exists, otherwise an empty Optional.

3. findAll()

Retrieves every student record and stores them in an ArrayList.

4. update()

Updates the register number, name, and email of an existing student.

5. deleteById()

Deletes the student record using the primary key.

The mapStudent() method converts database records into Java Student objects.

All queries use PreparedStatement, and try-with-resources automatically closes JDBC resources.

4.8 Java Program 5 – Main.java

Expected Output

Assuming the database is empty before execution:

Student inserted successfully.

Generated Student ID: 1

Student found:

Student ID: 1, Register No: 192311001, Name: Arun Kumar, Email: arun@gmail.com

All Students:

Student ID: 1, Register No: 192311001, Name: Arun Kumar, Email: arun@gmail.com

Student updated: true

Student deleted: true

The generated student ID may differ depending on the database's existing records.

5. Task A3.3 – Map Objects to Tables (1 Mark)

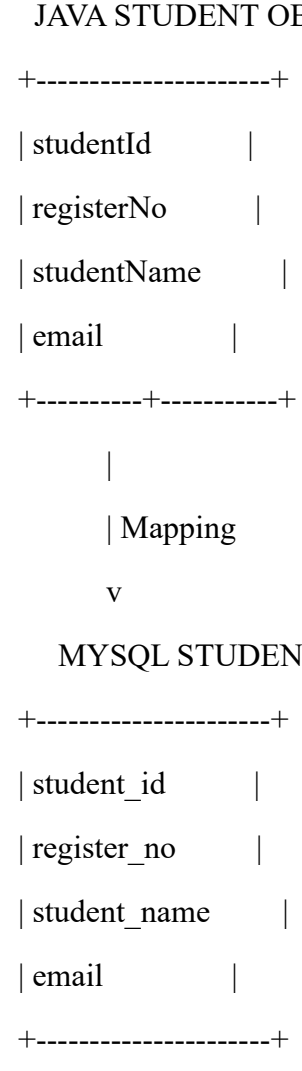
5.1 Student Class to Student Table Mapping

The Student class represents one row in the students table.

Each Java field corresponds to one database column.

Java Class Field	Database Column	SQL Data Type	Purpose
studentId	student_id	BIGINT	Primary key
registerNo	register_no	VARCHAR(20)	Student register number
studentName	student_name	VARCHAR(100)	Student name
email	email	VARCHAR(150)	Email address

5.2 Object-to-Table Mapping Diagram



5.3 Primary Key Mapping

The studentId field maps to the student_id column.

The corresponding SQL definition is:

student_id BIGINT AUTO_INCREMENT PRIMARY KEY

The primary key uniquely identifies each student record.

When a new student is inserted, MySQL automatically generates the student ID.

Java retrieves this generated value using:

```
ResultSet rs = ps.getGeneratedKeys();


if (rs.next()) {


   student.setStudentId(rs.getLong(1));
}
```

5.4 Example Object and Database Record

Java Object:

```
Student s1 = new Student(
   "192311001",
   "Arun Kumar",
   "arun@gmail.com"
);
```

Corresponding Database Record:

student_id	register_no	student_name	email
1	192311001	Arun Kumar	arun@gmail.com

This demonstrates how an object is stored as a row in a database table.

6. Alternative Concept and Complexity Analysis

6.1 Alternative – Writing SQL Directly in Service or GUI Classes

Instead of using a generic Dao interface, SQL queries can be written directly inside service or GUI classes.

For example:

```
String sql =
   "SELECT * FROM students WHERE student_id = ?";
```

```
Connection con = DatabaseConnection.getConnection();


PreparedStatement ps = con.prepareStatement(sql);


ps.setLong(1, studentId);


ResultSet rs = ps.executeQuery();
```

The same type of code may be repeated in multiple application classes.

Problems With This Approach

1. SQL queries become scattered across the application.
2. Database operations are mixed with business logic.
3. A database schema change requires modifications in multiple places.
4. Testing becomes difficult.
5. Code duplication increases.

6.2 Comparison When a Table Column Is Renamed

Suppose the database column:

student_name

is renamed to:

full_name

Without DAO:

If SQL queries are written directly in the GUI and service classes, every query using student_name must be updated.

For example:

- Student registration screen.
- Student search service.
- Student update service.
- Student report generation.

Several files may require modification.

With DAO:

The SQL statement and ResultSet mapping are maintained inside StudentDao.

Therefore, the database column rename can usually be handled by modifying the SQL and mapping code in one DAO class.

The service and GUI classes can continue using the same Student object fields and DAO method names.

6.3 Comparison Table

Feature	SQL Directly in GUI/Service	Generic DAO Design
SQL location	Multiple classes	DAO classes
Code duplication	Higher	Lower
Maintainability	Difficult	Easier
Database schema changes	Multiple locations may change	Usually limited to the affected DAO
Testing	GUI and database logic are connected	DAO can be tested separately
Reusability	Lower	Higher
Separation of responsibilities	Poor	Good

6.4 Time Complexity Analysis

The DAO design does not change the actual database query or its execution plan.

Therefore, both approaches have the same asymptotic time complexity for equivalent SQL operations.

For example:

Find Student by Primary Key

SELECT * FROM students WHERE student_id = ?;

With a primary-key index, the database can locate the record efficiently. For a typical B-tree index, lookup is approximately O(log n), subject to the database engine and execution plan.

Find All Students

SELECT * FROM students;

Retrieving all n students requires processing n records.

Time complexity:

O(n)

The generic DAO does not make the database query faster. It improves the organisation of the code.

6.5 Space Complexity Analysis

Both approaches require memory for:

- Database connections.
- PreparedStatement objects.
- ResultSet objects.
- Retrieved Java objects.

For retrieving one student:

O(1)

For retrieving all n students into an ArrayList:

O(n)

The DAO design introduces a small amount of additional abstraction, but it does not change the asymptotic space complexity of the database operations.

6.6 Engineering Decision

The generic DAO approach is recommended because it separates database operations from GUI and business logic.

It reduces duplicated code and makes the application easier to test, modify, and extend.

If a database table or column changes, the corresponding DAO implementation can be updated without changing every part of the application.

7. Final Result

The repeated database-access operations in StudentDB, CourseDB, and MarksDB were identified.

A generic Dao<T, K> interface was designed with insert, findById, findAll, update, and delete operations.

The StudentDao class was implemented to perform student-related database operations using JDBC and PreparedStatement.

The Student class was mapped to the MySQL students table, including the primary-key relationship.

Finally, the direct-SQL approach was compared with the generic DAO design based on maintainability, time complexity, and space complexity.

Thus, the DAO design pattern provides a reusable and maintainable approach to database access in Java applications.